

Plot Real Plane Curve Topology For Bivariate Polynomial

An Internship Report

Dai Yuejia

Supervisor:

Weijia Wang

August 6, 2026

Abstract

For a nonzero polynomial $P \in \mathbb{Q}[x, y]$, the real zero set $C(P) = \{(x, y) \in \mathbb{R}^2 : P(x, y) = 0\}$ may contain singularities, isolated points, vertical components, compact loops, and unbounded branches. A numerical contour plot can suggest its shape, but it does not certify how the branches meet. This internship developed a Julia package that replaces a purely visual description by a finite, exact, and testable graph representation.

The implementation first removes repeated factors and decomposes the input as $P(x, y) = V(x)H(x, y)$, where V records vertical components and H is primitive with respect to y . It computes projection coordinates from elimination, isolates real algebraic points in rational boxes, and matches each box to an exact critical abscissa. Ordinary rational fibers are then inserted between critical fibers. The order of the real roots on a regular strip determines the ordinary edges, while horizontal separator levels and interval refinement certify the incidence of branches at a critical fiber. A rational coordinate change handles projection directions in which branches may escape at a finite abscissa. The resulting graph retains isolated vertices and can be plotted or compactified by a single vertex at infinity. Euler's formula on the sphere then gives the number of connected components of $\mathbb{R}^2 \setminus C(P)$.

The final package is organized around exact isolation, topology construction, plotting, and region counting. Its present test suite contains fifteen passing assertions, including circles, a line, a hyperbola, a coordinate cross, and an isolated real point.

Keywords. Real algebraic curve; exact algebraic number; resultant; root isolation; topology graph; certified computation; Euler characteristic; Julia.

Contents

Abstract	i
1 Introduction	1
2 Mathematical Foundations	1
2.1 Input model and squarefree reduction	1
2.2 Resultants and projection coordinates	2
2.3 Algebraic numbers and isolating boxes	2
2.4 Root continuity inside a regular strip	3
2.5 Local separation at a critical fiber	3
2.6 Compactified graphs and Euler’s formula	3
3 From Rational Prototype to Certified Implementation	4
3.1 The rational prototype	4
3.2 Why the prototype was not the final interface	4
3.3 Architecture of the active package	4
3.4 Public data flow	4
4 Exact Isolation of Critical Fibers	5
4.1 Overview	5
4.2 Vertical decomposition and projection polynomial	5
4.3 Block elimination	6
4.4 Certified coordinate matching	6
4.5 Rational isolating intervals	6
4.6 Matching boxes to exact critical abscissas	6
4.7 Isolation algorithm summary	7
5 Certified Topology Graph Construction	7
5.1 Graph data model	7
5.2 Ordinary sample fibers	7
5.3 Edges inside regular strips	8
5.4 Certified incidence at a critical fiber	8
5.5 Vertical components and isolated points	8
5.6 Safe rational coordinates	8
5.7 Complete topology algorithm	9
6 Visualization and Plane Region Counting	9
6.1 Topology plotting	9
6.2 Compactification at infinity	9
6.3 Counting graph components and regions	9
7 Implementation and Validation	10
7.1 Principal public interfaces	10
7.2 Automated tests	10
7.3 Representative exact invariants	11
7.4 Correctness mechanisms visible in the source	13
7.5 Current limitations	13
8 Conclusion	14

1 Introduction

A bivariate polynomial is finite symbolic data, but its real zero set is a continuous geometric object. Even low-degree examples exhibit qualitatively different behavior: a circle is compact, a hyperbola has two unbounded components, $xy = 0$ contains a singular crossing, and $x^2 + y^2 = 0$ consists of a single isolated real point. A topology algorithm must distinguish these cases without relying on the visual resolution of a plot.

The central objective of the internship was to implement the following chain: starting from $P \in \mathbb{Q}[x, y]$, compute exact finite data above its special vertical fibers; add regular sample fibers; connect all branches with explicit certificates; draw the resulting graph; and count the connected regions in the complement. The active implementation is the Julia package `PlotCurveTopologyForBivariatePolynomial`. Its public entry points expose each stage separately, so intermediate results can be inspected instead of being hidden inside one monolithic plotting command.

The work followed two development stages. The preserved rational prototype makes the geometry visible through a hand-written Sylvester matrix, rational root intervals, sample points, and an adjacency matrix. The active method uses Nemo algebraic numbers, block elimination from `AlgebraicSolving`, Arb ball arithmetic, separating linear projections, and rational isolating boxes. This transition was not simply a change of numerical precision: it replaced heuristic coordinate matching by exact equality and certified separation.

Main outcomes. The repository provides:

- exact isolation of all finite topology data on special vertical fibers, including intersections with vertical components;
- explicit matching between rational boxes and exact critical abscissas;
- a topology graph that retains both edges and degree-zero vertices;
- certified left and right branch incidence at every critical fiber;
- plotting in a safe coordinate direction and mapping back to the original coordinates;
- compactification at infinity and region counting by Euler's formula.

Evidence boundary. This report is reconstructed from the current source tree, the preserved prototype, the four proof notes, the examples, and the executable tests. No benchmark section is included because the repository does not contain a controlled performance study. Consequently, the validation claims concern functional behavior and exact invariants rather than speedups.

2 Mathematical Foundations

2.1 Input model and squarefree reduction

The package accepts a nonzero polynomial $P \in \mathbb{Q}[x, y]$. Repeated factors do not change the geometric set $C(P)$, so the active code first replaces P by its squarefree part

$$P_{\text{sf}} = \frac{P}{\gcd(P, P_x, P_y)}.$$

The implementation computes the greatest common divisor successively with the derivative in each variable. This step is important because the discriminant of a polynomial with repeated factors would otherwise vanish for reasons unrelated to the geometry of distinct branches.

The squarefree polynomial is then decomposed as

$$P_{\text{sf}}(x, y) = V(x)H(x, y), \quad (1)$$

where V is the monic content of P_{sf} with respect to y , and H is primitive in $\mathbb{Q}[x][y]$. Every real root α of V defines a vertical component $x = \alpha$. The primitive part H contains the remaining curve and cannot vanish identically on an ordinary rational vertical fiber.

2.2 Resultants and projection coordinates

Let $F, G \in K[t]$ have degrees m and n . Their Sylvester matrix is the $(m+n) \times (m+n)$ matrix formed from shifted coefficient vectors of F and G , and the resultant is

$$\text{Res}_t(F, G) = \det(\text{Syl}(F, G)).$$

Proposition 2.1 (Resultant criterion). *For $F, G \in K[t]$, where K is a field,*

$$\text{Res}_t(F, G) = 0 \iff \gcd(F, G) \notin K[t]^\times.$$

Proof. The Sylvester matrix represents the linear map $(A, B) \mapsto AF + BG$ under the degree bounds $\deg A < n$ and $\deg B < m$. Its determinant is zero exactly when a nonzero pair (A, B) lies in the kernel. If F and G have a nonconstant common factor, the complementary quotients produce such a pair. Conversely, if $AF + BG = 0$ under the degree bounds and F and G were coprime, Euclid's lemma would imply $F \mid B$, contradicting $\deg B < m$ unless $B = 0$, and then also $A = 0$. This is the criterion developed in the corresponding proof note. $\square$

For the primitive part $H(x, y)$, a vertical critical point satisfies $H = H_y = 0$. Eliminating y therefore produces a univariate polynomial whose real roots contain the corresponding critical abscissas. The package also includes the roots of $\text{lc}_y(H)$ because a drop of degree in y may indicate a branch that tends to infinity above a finite x -coordinate. The projection set used by the proof notes is

$$\text{proj}(H) = \{\alpha \in \mathbb{R} : \text{Disc}_y(H)(\alpha) = 0 \text{ or } \text{lc}_y(H)(\alpha) = 0\}.$$

2.3 Algebraic numbers and isolating boxes

Definition 2.2. *An algebraic number is a complex root of a nonzero polynomial in $\mathbb{Q}[t]$. A real algebraic number can be represented exactly together with a rational interval that contains it and excludes every other root of its defining polynomial.*

The proof notes show that algebraic numbers are closed under addition, multiplication, additive inverses, and multiplicative inverses. This closure is essential for exact coordinate transformations and substitutions. In the active code, Nemo's algebraic closure of $\mathbb{Q}$ represents the exact values. A bisection loop produces a rational interval of width at most 2^{-p} for a requested precision p .

For a point $(\alpha, \beta) \in \mathbb{R}^2$, the finite representation returned to the rest of the package is an isolating box

$$I_x \times I_y = (x_\ell, x_r) \times (y_\ell, y_r),$$

with rational endpoints. The code additionally retains exact algebraic coordinates internally when equality or ordering must be certified.

2.4 Root continuity inside a regular strip

Let $\alpha_1 < \dots < \alpha_s$ be the real projection coordinates of a squarefree polynomial H with positive y -degree. The vertical strips between consecutive coordinates, including the two outer strips, contain no zero of $\text{Disc}_y(H) \text{lc}_y(H)$.

Proposition 2.3 (Constancy of real roots). *On every open regular strip (α_i, α_{i+1}) , the number of distinct real roots of $H(x, Y)$ is constant. These roots extend to continuous branches that preserve their vertical order.*

Proof. The leading coefficient does not vanish inside the strip, so every fiber is a nonzero polynomial of fixed degree in Y . The discriminant does not vanish, so every real root is simple. The implicit function theorem therefore gives a unique local C^1 branch through each real point. A branch cannot stop at an interior abscissa: its values remain locally bounded because the leading coefficient stays away from zero, and a limiting root can again be continued by the implicit function theorem. Local uniqueness pastes the continuations into a branch across the strip. Distinct branches cannot cross without creating a multiple root. Applying the continuation argument in both directions shows that the root count is the same on any two fibers. This is the argument developed in the regular-strip proof note. $\square$

The proposition justifies a simple but powerful rule: on two ordinary fibers in the same strip, the k -th root from the bottom belongs to the same branch.

2.5 Local separation at a critical fiber

Suppose that the finite roots on $x = \alpha$ lie in disjoint boxes ordered from bottom to top. Choose rational horizontal levels between consecutive boxes, and one additional level below the first and above the last.

Proposition 2.4 (Horizontal separation). *Each chosen level can be kept free of the curve in a sufficiently narrow slab around $x = \alpha$. Consequently, every nearby branch remains in a unique horizontal window containing one critical box.*

Proof. For a separator height b , the construction ensures $H(\alpha, b) \neq 0$. The function $x \mapsto H(x, b)$ is continuous, hence it stays nonzero on some interval $|x - \alpha| < \varepsilon_b$. Taking the minimum over the finite set of separator heights gives one slab in which no separator intersects the curve. This is the local argument recorded in the branch-separation proof note. $\square$

The implementation strengthens this existence statement into an explicit certificate: it computes all real roots of each horizontal slice $H(x, b)$ and refines the isolating interval of α until none of those roots lies in the interval.

2.6 Compactified graphs and Euler's formula

The finite sampled graph stops unbounded branches at outer sample fibers. To count regions, the package compactifies the plane to a sphere and joins every unbounded branch end to one additional vertex ∞ . Let the resulting embedded multigraph have V vertices, E edges, and C connected components. Euler's formula gives

$$F = E - V + C + 1, \quad (2)$$

where F is the number of connected regions in $\mathbb{R}^2 \setminus C(P)$. Parallel edges are deliberately preserved because they may represent different curve arcs and each contributes to E .

3 From Rational Prototype to Certified Implementation

3.1 The rational prototype

The directory `prototype/rational_method/` preserves the original development sequence as standalone scripts. Its main components are:

- extraction of polynomial coefficients and construction of a Sylvester matrix;
- determinant-based resultants and squarefree reduction;
- real-root isolation based on a root bound, sign variations, interval subdivision, and a Mobius transformation;
- selection of critical and ordinary sample fibers;
- an adjacency matrix connecting ordinary and critical sample points;
- conversion of the matrix to a graph for visualization.

This version made the intended geometric pipeline concrete. In particular, it established that one does not need to sample the entire plane: it is enough to identify special vertical fibers, insert ordinary fibers between them, and connect compatible roots.

3.2 Why the prototype was not the final interface

The prototype files intentionally contain duplicated helpers and diagnostic output. They are excluded from the package module so that historical variants cannot overwrite the active definitions. Several design changes were required for the current implementation:

1. projected coordinates need not be rational, so a rational-only data model cannot represent every critical fiber exactly;
2. nearest-point matching does not by itself certify that two samples lie on the same branch;
3. an edge-only graph loses isolated real points;
4. vertical components and branches escaping to infinity require separate treatment;
5. root isolation, topology construction, plotting, and region counting benefit from independent interfaces and tests.

3.3 Architecture of the active package

The active module loads six focused source files. The organization mirrors the mathematical pipeline rather than the chronological order of the prototype.

The package declaration constrains Julia to version 1.10 or later and records Nemo 0.56, AlgebraicSolving 0.11, and Plots 1 as its direct dependencies. Nemo supplies polynomial arithmetic, algebraic numbers, factorization, resultants, and Arb balls. AlgebraicSolving supplies block elimination, while Plots is restricted to the presentation layer.

3.4 Public data flow

The active implementation exposes intermediate products instead of only a final image. A caller can request raw isolation boxes, matched critical boxes, all sample boxes, the full topology graph, a plot, the compactified graph, or the final region count. This separation made the three test groups possible: isolation can fail independently of connection, and graph construction can be checked independently of visualization.

Table 1: Active source organization.

Source file	Responsibility
src/isolation/bivariate_real_isolation.jl	Squarefree reduction, vertical decomposition, elimination, exact point matching, and rational isolating boxes.
src/isolation/qbar_critical_points.jl	Critical abscissas from the leading coefficient and discriminant resultant, represented in $\overline{\mathbb{Q}}$.
src/isolation/match_critical_boxes.jl	Containment-based matching between bivariate boxes and disjoint critical intervals.
src/topology/connect_critical_boxes.jl	Ordinary fibers, certified incidence at critical fibers, vertices, and edges.
src/plotting/plot_critical_box_edges.jl	Safe coordinates, inverse mapping, and display of topology graphs.
src/topology/count_plane_regions.jl	Compactification, graph components, and Euler region counts.

4 Exact Isolation of Critical Fibers

4.1 Overview

The exact isolation stage converts an infinite curve into finite algebraic data on special fibers. The implementation does not attempt to isolate every point of $P = 0$; it isolates precisely the points needed by the topology stage.

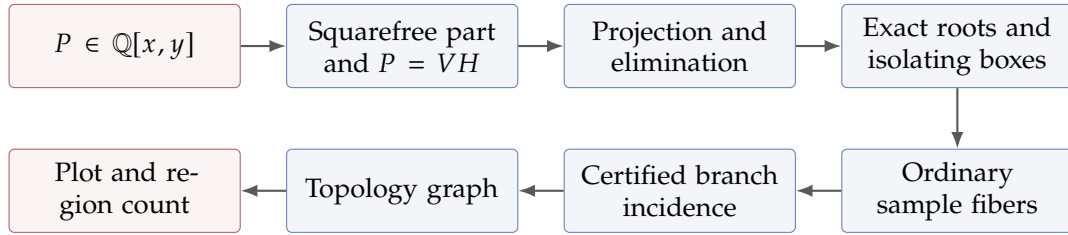


Figure 1: Data flow of the active certified implementation.

4.2 Vertical decomposition and projection polynomial

After computing P_{sf} and the decomposition in [equation \(1\)](#), the main isolation routine handles two finite systems.

First, if $\deg_y(H) > 0$, it computes H_y and an elimination polynomial $C(x)$ for the ideal generated by H and H_y . It then forms

$$E(x) = \text{sqfree}(C(x) \text{lc}_y(H)(x)).$$

Solving $H(x, y) = E(x) = 0$ returns every finite point of $H = 0$ above a critical or degree-dropping abscissa, including noncritical points on the same special fiber. The entire fiber is retained because all of its finite roots are needed for branch ordering.

Second, if V has real roots, the system $V(x) = H(x, y) = 0$ yields all intersections between vertical components and the primitive curve. Duplicate points belonging to both collections are removed before the boxes are returned.

4.3 Block elimination

The helper `eliminationPolynomial(F, G, keep)` reorders the two variables so that the variable to eliminate is the first block. It constructs the ideal $\langle F, G \rangle$, calls `AlgebraicSolving`'s block elimination, extracts univariate candidates in the retained variable, selects the candidate of smallest degree, and takes its squarefree part. If no univariate polynomial is produced, the system is reported as non-zero-dimensional rather than silently returning incomplete data.

The source also contains a direct Buchberger implementation and multivariate normal-form helpers. They document the algebraic operations explicitly, while the active elimination path uses `AlgebraicSolving` for the main computation.

4.4 Certified coordinate matching

Separate eliminants in x and y provide candidate coordinate sets, but the Cartesian product contains false pairs. The active specialized solver reduces this cost and certifies the correspondence as follows.

Algorithm 1 Certified coordinate matching for $H(x, y) = q(x) = 0$

1. Replace q by its squarefree part and factor it over $\mathbb{Q}$.
 2. For each irreducible factor, compute its exact real roots $x_1, \dots, x_r$.
 3. Eliminate x locally to obtain the possible real y -coordinates $y_1, \dots, y_s$.
 4. Evaluate H on Arb balls around each (x_i, y_j) . Discard a pair only when the resulting ball excludes zero.
 5. Search for a nonzero integer λ such that $t = x + \lambda y$ separates all retained solutions.
 6. Compute the projection polynomial in t and a linear relation $A(t)x + C(t) = 0$ from a primitive pseudo-remainder sequence.
 7. Increase ball precision until every real root τ of the projection polynomial determines one unique candidate pair through $x = -C(\tau)/A(\tau)$ and $y = (\tau - x)/\lambda$.
-

The implementation tries $\lambda = \pm 1, \dots, \pm 32$. If no projection is certified, it retains a local exact-substitution fallback after factorization and Arb rejection have already reduced the candidate set. Thus Arb arithmetic is used only for certified exclusion and disjointness; final equality is never decided by a floating-point tolerance.

4.5 Rational isolating intervals

For every exact real algebraic coordinate a , the function `isolateAlgebraicNumber` first doubles an integer bound until $-B \leq a \leq B$, then bisepts rationally until the interval width is at most 2^{-p} . If a is exactly equal to a rational midpoint, the function can return a degenerate interval (a, a) . Otherwise it returns two rational endpoints enclosing a .

The output of `bivariateRealIsolation(P, p)` is a sorted set of boxes

$$((x_\ell, x_r), (y_\ell, y_r)).$$

Only finite topology data is returned. An entire vertical component cannot be stored as a finite set of boxes, so the decomposition retains V separately for the topology stage.

4.6 Matching boxes to exact critical abscissas

The independent critical-coordinate path computes

$$\text{roots}_{\mathbb{R}}(\text{lc}_y(P_{\text{sf}})) \cup \text{roots}_{\mathbb{R}}(\text{Res}_y(P_{\text{sf}}, (P_{\text{sf}})_y)).$$

Each exact coordinate α receives a rational interval. The box-matching routine verifies that these intervals are ordered and pairwise disjoint, then requires every box's x -interval to be contained in exactly one critical interval. Its output has the form

$$((\alpha, I_\alpha), I_y),$$

combining an exact algebraic abscissa with rational intervals suitable for refinement and plotting. Missing or ambiguous containment is treated as an error, normally indicating insufficient precision.

4.7 Isolation algorithm summary

Algorithm 2 Special-fiber isolation for $P \in \mathbb{Q}[x, y]$

1. Validate that P is a nonzero polynomial in exactly two variables over $\mathbb{Q}$, and compute P_{sf} .
 2. Compute the monic vertical content $V(x)$ and primitive part $H(x, y)$.
 3. Form the special-abscissa polynomial from elimination of H, H_y and from $\text{lc}_y(H)$.
 4. Solve $H = 0$ above these abscissas using exact roots, Arb rejection, and a separating projection.
 5. Solve $V = H = 0$ for intersections with vertical components.
 6. Remove duplicates, isolate every coordinate rationally, and sort the resulting boxes.
 7. Compute exact critical abscissas independently and match each box to one disjoint critical interval.
-

5 Certified Topology Graph Construction

5.1 Graph data model

The main interface `topologyGraphData(P)` returns

$$(\text{vertices} = \mathcal{V}, \text{ edges} = \mathcal{E}).$$

Every vertex is a critical or ordinary sample box. An edge is an ordered pair of boxes representing one curve arc between consecutive sampled fibers or along a vertical component. Keeping $\mathcal{V}$ separately is essential: the isolated point of $x^2 + y^2 = 0$ has degree zero and therefore cannot be recovered from $\mathcal{E}$.

5.2 Ordinary sample fibers

Suppose the critical coordinates have disjoint rational isolating intervals $I_1 < \dots < I_s$. The code selects a configurable number of rational sample abscissas in each of the $s + 1$ open strips. Bounded-strip samples are rational fractions of the gap between consecutive critical intervals. The two outer strips receive samples in unit-width windows just outside the first and last critical intervals.

At an ordinary rational abscissa β , substitution produces a univariate polynomial $H(\beta, Y)$. Its exact real roots are sorted and isolated in y . This specialized path avoids running the general bivariate solver on every sample fiber.

5.3 Edges inside regular strips

By the root-constancy proposition, two ordinary fibers inside the same strip have the same number of real roots and the same vertical ordering. The code therefore connects the k -th box on one fiber to the k -th box on the next. If the root counts disagree, construction stops with an internal-consistency error instead of guessing a matching. Additional samples only subdivide the same branches and do not change their topology.

5.4 Certified incidence at a critical fiber

Let $B_1, \dots, B_t$ be the boxes on $x = \alpha$, ordered from bottom to top. The difficult step is to determine which roots immediately to the left and right approach each B_j .

The code constructs disjoint horizontal windows around the boxes. A candidate boundary is chosen rationally in every gap and is rejected if it is itself a horizontal component of H . It then computes the exact roots in x of every horizontal slice $H(x, b)$. The critical interval is successively refined until none of these roots lies over the interval. At that point no branch can cross a window boundary in the local slab, and the ordered roots on its left and right boundary can be assigned uniquely to windows.

Algorithm 3 Certified incidence at $x = \alpha$

1. Sort the critical boxes by the midpoint of their y -intervals and verify that the intervals are disjoint.
 2. Choose one rational horizontal boundary below, between, and above the boxes; reject a boundary for which $H(x, b)$ is identically zero.
 3. Compute all exact real roots of each boundary polynomial $H(x, b)$.
 4. Refine the isolating interval I_α until no boundary root lies in I_α .
 5. Evaluate the exact real roots on the left and right rational endpoints of I_α .
 6. Assign each root to the unique horizontal window containing it, and connect it to the corresponding critical box.
-

A projection coordinate caused only by a vanishing leading coefficient may have no finite box above it. In this case the adjacent branches are treated as unbounded at the finite abscissa and receive no false finite attachment.

5.5 Vertical components and isolated points

For every real root α of V , the package finds the intersections of $x = \alpha$ with $H = 0$ and connects consecutive intersection boxes from bottom to top. The two unbounded rays of a vertical component are not finite box-to-box edges; they are accounted for later during compactification.

Isolated real points require the opposite treatment: they are retained as vertices even though they have no incident edge. The separate vertex array is therefore not a cosmetic interface choice but part of the topological data.

5.6 Safe rational coordinates

If $\text{lc}_y(P)$ is nonconstant, a branch can escape to infinity above a finite x -coordinate. Both plotting and region counting first search for a safe rational direction. For an integer b , the code applies

$$x = u + bv, \quad y = -bu + v. \quad (3)$$

The determinant is $1 + b^2 > 0$, so this is an invertible linear map. If D is the total degree and P_D the highest homogeneous part, the coefficient of v^D in the transformed polynomial is $P_D(b, 1)$. Only finitely many integers b make this value zero. The code checks $0, -1, 1, -2, 2, \dots$ until the transformed polynomial has full v -degree and a nonzero constant leading coefficient. Exact graph vertices are mapped back through [equation \(3\)](#) before conversion to floating-point plotting coordinates.

5.7 Complete topology algorithm

Algorithm 4 Construction of the sampled topology graph

1. Prepare $P_{sf} = VH$, critical coordinates, critical boxes, ordinary sample abscissas, and all ordinary-fiber boxes.
 2. Connect equal-rank roots on consecutive ordinary fibers inside every regular strip.
 3. For each critical coordinate, certify horizontal windows and compute left and right branch incidence.
 4. Attach the nearest ordinary fibers to their certified critical boxes.
 5. Add edges between consecutive intersections on every vertical component.
 6. Sort vertices and edges while retaining degree-zero vertices and parallel edges.
-

6 Visualization and Plane Region Counting

6.1 Topology plotting

The plotting layer represents a box by its exact algebraic x -coordinate and the midpoint of its rational y -interval, converted to Float64 only at the final display step. Edges are drawn as straight segments between these representatives. They are not intended as a high-resolution approximation of the original curve; their purpose is to make the finite topology graph visible.

The function `plotCriticalBoxes` computes the full graph, applies the safe transformation when necessary, maps vertices and edges back to the original coordinates, and optionally saves the result. The complete vertex collection is passed to the scatter layer, so isolated points remain visible.

6.2 Compactification at infinity

The outermost ordinary fibers truncate every unbounded branch. In a safe coordinate direction, every root on the leftmost or rightmost sampled fiber represents exactly one unbounded end. The function `compactifiedTopologyGraphData` creates one value of type `PlaneInfinityVertex` and connects each such end to it. The infinity vertex is retained even when the curve is compact; in that case it is an isolated graph component representing the complement of the affine plane in the sphere.

6.3 Counting graph components and regions

The implementation counts connected graph components with a disjoint-set union structure using path compression and union by rank. It first validates that every edge endpoint belongs to the vertex collection. It intentionally does not deduplicate edges, because two arcs may have identical sampled endpoints while enclosing a region between them.

If the compactified graph has V vertices, E edges, and C components, `countPlaneRegions` returns the value in [equation \(2\)](#). The zero polynomial is handled separately: its zero set is the whole plane, so the complement has zero connected regions.

7 Implementation and Validation

7.1 Principal public interfaces

The module exports the functions in [table 2](#). Precision parameters are nonnegative, and graph construction requires the box precision to exceed the critical-interval precision so that each box fits inside its matched interval.

Table 2: Principal public interfaces.

Interface	Result
<code>bivariateRealIsolation(P,p)</code>	Rational boxes for finite special-fiber topology data.
<code>getPointsCritical(P)</code>	Exact real algebraic projection coordinates.
<code>criticalPointBoxes(P)</code>	Critical boxes matched to exact abscissas and isolating intervals.
<code>getTopologySampleBoxes(P)</code>	All critical and ordinary sample boxes used by the graph.
<code>topologyGraphData(P)</code>	Complete sampled vertices and box-to-box edges.
<code>plotCriticalBoxes(P)</code>	A Plots.jl visualization, optionally saved to a file.
<code>compactifiedTopologyGraphData(P)</code>	Finite graph augmented by a vertex at infinity.
<code>countPlaneRegions(P)</code>	Number of connected regions in $\mathbb{R}^2 \setminus C(P)$.

7.2 Automated tests

The test entry point loads three focused files. Running `Pkg.test()` on August 6, 2026 completed all fifteen assertions:

Table 3: Current automated test summary.

Test set	Passed	Total
Exact isolation	3	3
Topology graph	5	5
Complement region counts	7	7
Total	15	15

The isolation tests verify that a unit circle has two critical abscissas, two special boxes, and two matched critical boxes. The graph tests verify four vertices and four edges for the sampled circle, retain one isolated vertex and no edges for $x^2 + y^2 = 0$, and confirm that the plotting interface is available. The region tests cover two nested circles, a line, a hyperbola, a coordinate cross, an isolated point, and the zero polynomial.

7.3 Representative exact invariants

For the nonzero examples, [table 4](#) records the graph after compactification. The counts were produced by the current package with `boxPrecision=24`, `intervalPrecision=12`, and one sample per strip. The component count follows from the same vertex and edge collection used by the Euler calculation.

Table 4: Validated compactified graph invariants and complement regions.

Polynomial	V	E	C	Regions
$x^2 + y^2 - 1$	5	4	2	2
$(x^2 + y^2 - 1)(x^2 + y^2 - 4)$	17	16	3	3
y	3	3	1	2
$xy - 1$	7	8	1	3
xy	6	8	1	4
$x^2 + y^2$	2	0	2	1

The infinity vertex explains the apparently extra graph component for a compact curve. For the circle, for example, the sampled cycle is one component and the isolated infinity vertex is a second component; hence $4 - 5 + 2 + 1 = 2$ regions. For the isolated point, both the finite point and infinity are isolated vertices, so $0 - 2 + 2 + 1 = 1$.

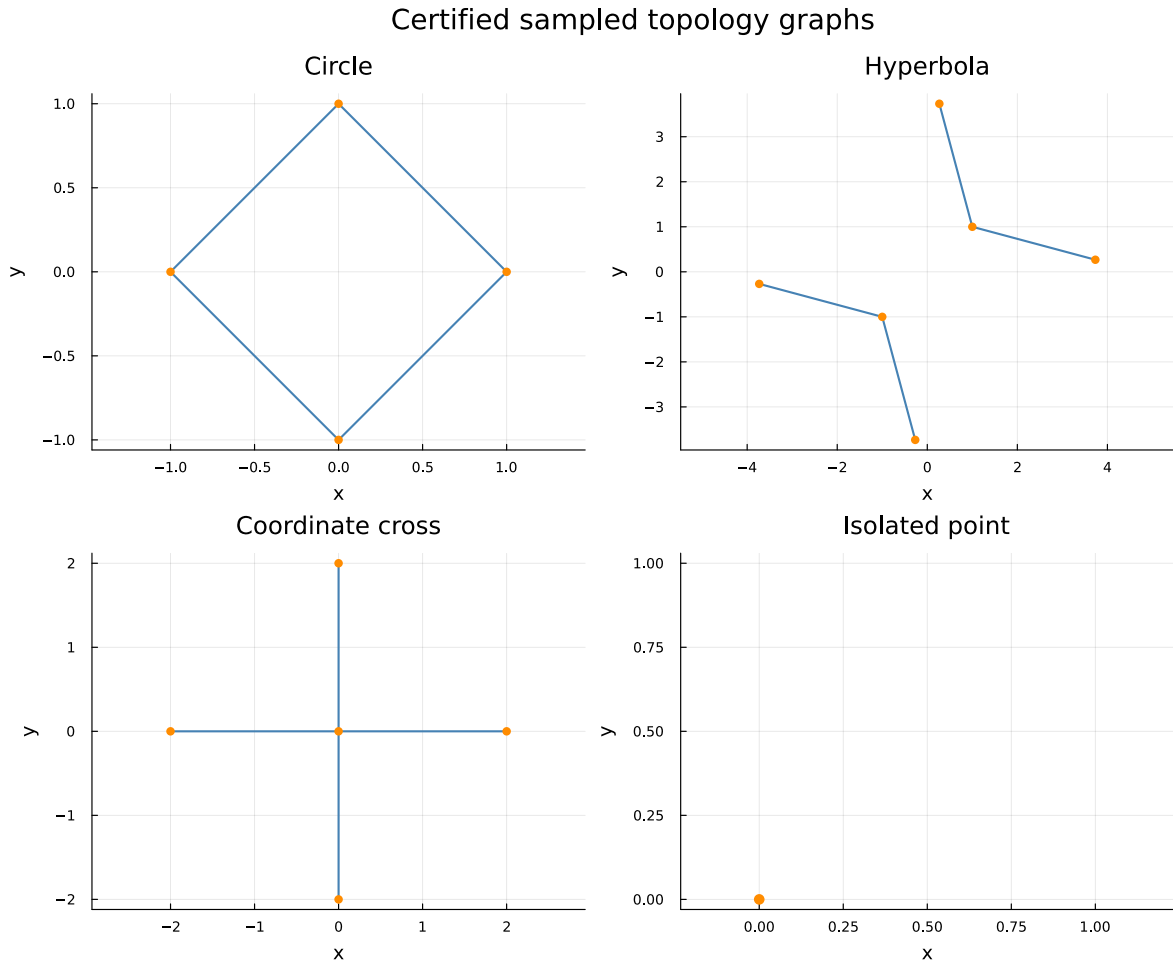


Figure 2: Topology-graph visualizations generated by the current package. The straight segments represent certified sampled adjacency, not a numerical contour approximation.

7.4 Correctness mechanisms visible in the source

The active code contains explicit guards at the points where a heuristic implementation would commonly make an unchecked assumption:

- input rings, variable counts, nonzero conditions, and precision parameters are validated;
- critical intervals must be ordered and pairwise disjoint;
- every bivariate box must match exactly one critical interval;
- y-intervals on a critical fiber must be disjoint;
- ordinary fibers in one strip must have equal root counts;
- horizontal separators must be certified clear of the local slab;
- a separating projection must produce a bijection with candidate coordinate pairs;
- all graph edge endpoints must occur in the vertex collection.

These checks do not replace a complete formal proof of the program, but they turn the main mathematical preconditions into executable failure conditions.

7.5 Current limitations

The present scope is deliberately narrower than a general-purpose real algebraic geometry library:

- the public input model is restricted to $\mathbb{Q}[x, y]$;
- exact elimination and algebraic-number comparisons may dominate the cost for high-degree or dense inputs;
- the specialized separating projection tries a finite list of small integer slopes before using exact substitution as a fallback;
- local incidence may require higher box precision or more interval refinements on difficult inputs;
- the automated suite covers representative geometries but remains small and contains no randomized regression corpus;
- the repository contains no controlled timing benchmark, so no performance comparison is claimed in this report.

8 Conclusion

The internship produced an end-to-end implementation for real plane algebraic curves over $\mathbb{Q}$. The final workflow begins with squarefree reduction and vertical decomposition, obtains exact projection coordinates, isolates finite special points in rational boxes, certifies branch incidence, builds a topology graph, draws the graph in the original coordinates, and counts complement regions after compactification. The development path from a rational prototype to an exact package is preserved in the repository and explains the major interface decisions of the active code.

The most important design principle is the separation of exact computation from presentation. Algebraic equality, ordering, exclusion, and incidence are resolved before any conversion to floating point. Plotting then becomes a view of already-computed topology rather than evidence for it. The explicit vertex collection likewise separates the mathematical graph from an edge-only visualization and makes isolated points first-class results.